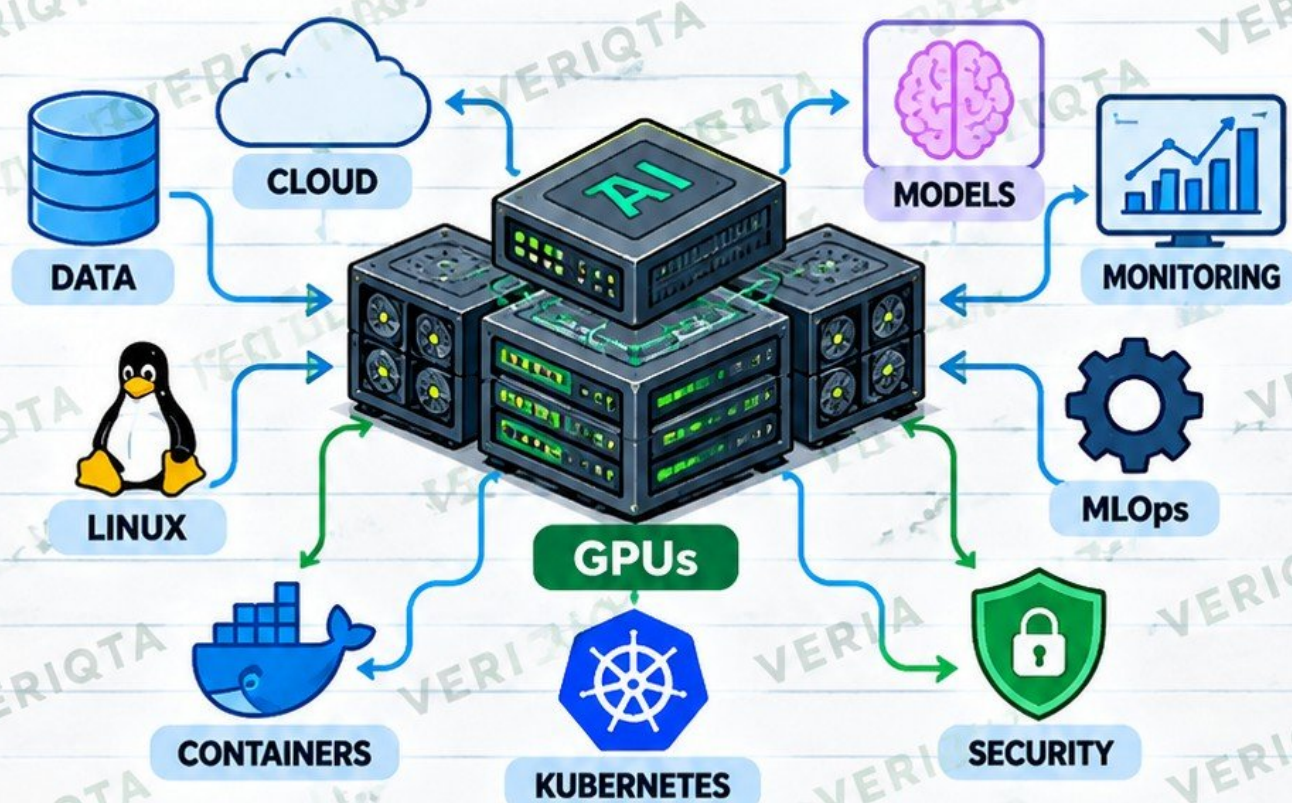


AI INFRASTRUCTURE ENGINEERING FUNDAMENTALS

A PRACTICAL GUIDE FOR BEGINNERS



LEARN | BUILD | PRACTICE | GROW

01. INTRODUCTION TO AI INFRASTRUCTURE

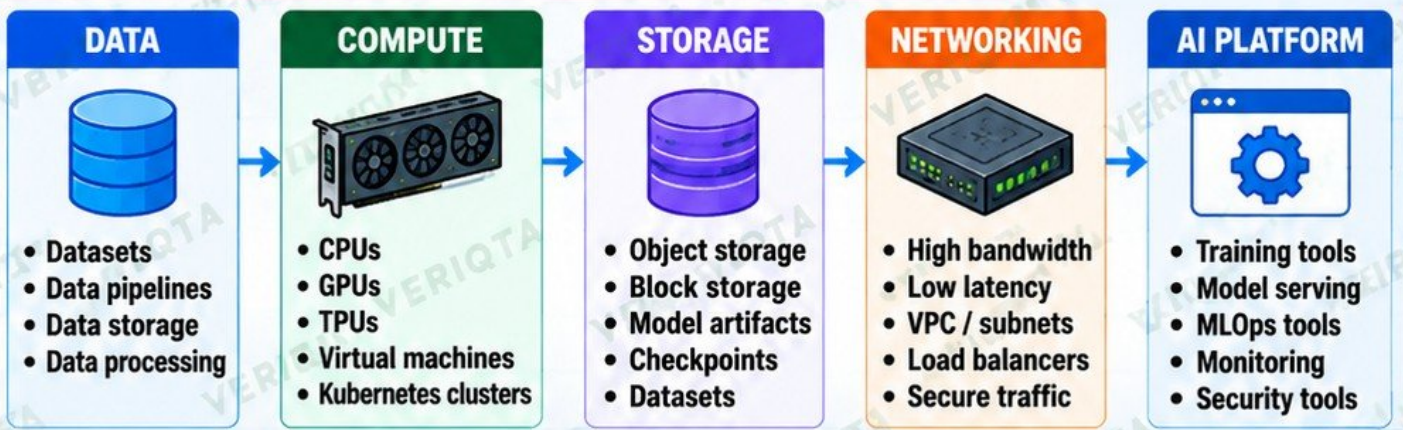
WHAT IS AI INFRASTRUCTURE?

- **AI infrastructure** is the combination of hardware, software, storage, networking and tools required to develop, train, deploy and operate AI models and applications.
- It provides the computing power, data storage, networking and platforms needed to run AI workloads reliably and at scale.

WHY DOES IT EXIST?

- AI models require **large amounts of data** and **high computing power**, especially GPUs.
- It solves the problem of **training and running complex models efficiently**.
- It enables faster development, lower training time and reliable deployment of AI applications.

AI INFRASTRUCTURE COMPONENTS (HIGH LEVEL)



WHERE IS IT USED?

- Training large language models (LLMs)
- Running AI applications and APIs
- Computer vision, speech and recommendation systems
- Research and experimentation
- Enterprise AI solutions in cloud and on-premises

ENGINEER'S RESPONSIBILITIES

- Provision and manage compute, storage and networking
- Set up and maintain environments for training and serving
- Automate deployment and scaling
- Monitor performance, cost and reliability
- Implement security and access controls
- Troubleshoot infrastructure and model issues
- Work with data engineers, ML engineers and application teams

BEGINNER MISTAKES

- ✗ Thinking AI is just a model, not infrastructure
- ✗ Ignoring compute and storage requirements
- ✗ Underestimating the cost of training
- ✗ Not planning for monitoring and security
- ✗ Assuming local machines are enough

KEY TAKEAWAY

- ✓ AI infrastructure is the foundation for AI systems.
- ✓ It includes compute, storage, networking, platforms and tools.
- ✓ Engineers play a key role in building, operating and maintaining this infrastructure.
- ✓ A good understanding of infrastructure is essential to run AI workloads reliably, securely and cost-effectively.

02. AI INFRASTRUCTURE ARCHITECTURE

Understand the relationship between compute, storage, networking, data, models and applications.

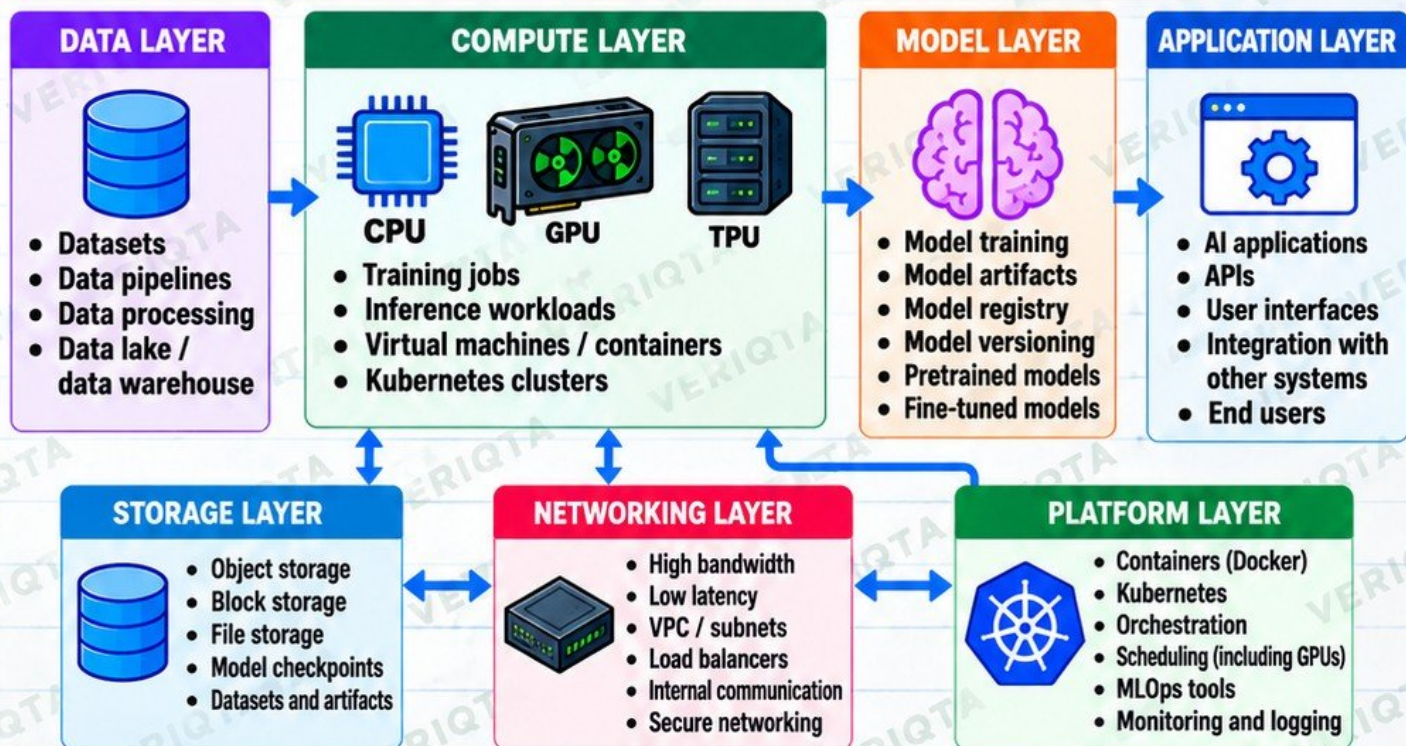
WHAT IS AI INFRASTRUCTURE ARCHITECTURE?

- It is the design of the **compute, storage, networking, data, models and platforms** that work together to develop, train, deploy and run AI applications.
- It shows how all the components connect and support each other.

WHY DOES IT MATTER?

- AI workloads require **large amounts of compute, storage and data**.
- A good architecture ensures high performance, reliability, security and scalability.
- It helps you run AI applications efficiently and control costs.

AI INFRASTRUCTURE ARCHITECTURE (HIGH LEVEL)



HOW THE COMPONENTS WORK TOGETHER

- Data is stored and processed in the data layer.
- Compute resources run training or inference jobs.
- Models are trained, stored and versioned.
- Applications use the models through APIs.
- Networking connects all components.
- The platform layer manages and orchestrates everything.

REAL WORLD EXAMPLE

- A company collects and processes data for a chatbot.
- GPUs in a Kubernetes cluster train a language model.
- The trained model is stored in object storage and a model registry.
- The model is deployed as an API.
- Users access the chatbot through a web or mobile application.

KEY TAKEAWAYS

- AI infrastructure is a connected system.
- Compute, storage, networking, data, models and applications work together.
- A good architecture improves performance, reliability, security and cost efficiency.
- Engineers need to understand how each component fits into the overall system.

03. THE AI WORKLOAD LIFECYCLE

Follow an AI workload from data preparation to training, deployment, inference, and monitoring.

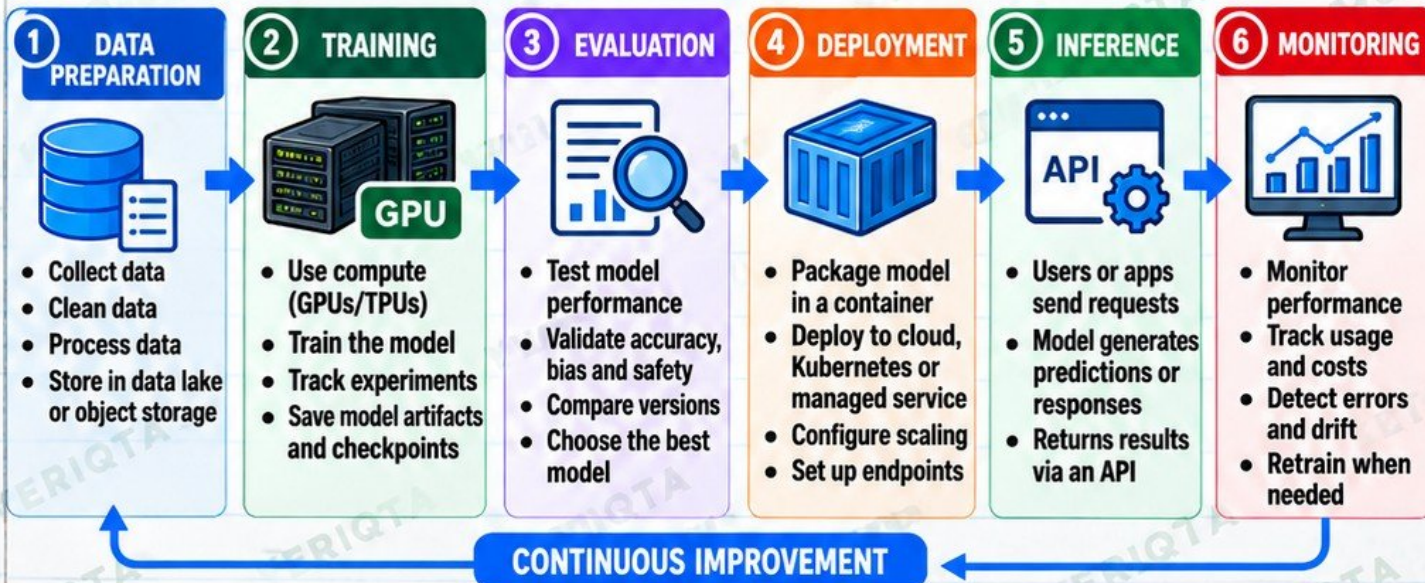
WHAT IS THE AI WORKLOAD LIFECYCLE?

- It is the **end-to-end process** of building, training, deploying and operating AI models.
- It includes **data preparation, training, evaluation, deployment, inference** and continuous monitoring.
- Each stage relies on infrastructure such as compute, storage, networking and platforms.

WHY DOES IT MATTER?

- Helps you understand how AI models go from **raw data** to **real-world applications**.
- Allows better planning of **infrastructure, cost and resources**.
- Helps you identify where problems can occur and how to troubleshoot.
- It enables **reliable, secure and scalable** AI applications in production.

AI WORKLOAD LIFECYCLE (HIGH LEVEL)



INFRASTRUCTURE USED AT EACH STAGE

- Compute:** CPUs, GPUs, TPUs, virtual machines, Kubernetes clusters
- Storage:** Object storage, block storage, datasets, model artifacts, checkpoints
- Networking:** VPC, subnets, load balancers, high bandwidth, low latency
- Platforms:** Docker, Kubernetes, cloud services, MLOps tools

COMMON CHALLENGES

- Poor quality data
- Not enough compute resources
- High training costs
- Slow training or inference
- Model underperformance
- Deployment failures
- Scaling issues
- Monitoring and detecting model drift
- Security and access control

KEY TAKEAWAYS

- AI workloads follow a clear lifecycle from data to monitoring.
- Each stage has different infrastructure requirements.
- Engineers play a key role in building, operating and maintaining the infrastructure.
- Continuous monitoring and improvement keep AI applications reliable, secure and cost-effective.

04. CPU vs GPU vs TPU

Understand processing differences and why AI workloads require specialized compute.

WHAT ARE THEY?

- **CPU** (Central Processing Unit)
General-purpose processor for all types of computing tasks.
- **GPU** (Graphics Processing Unit)
Specialized processor designed for parallel processing.
- **TPU** (Tensor Processing Unit)
Google's custom chip designed specifically for AI and machine learning workloads.

WHY ARE THEY DIFFERENT?

- **CPUs** are optimized for sequential processing and handle a wide range of tasks.
- **GPUs** are optimized for parallel processing, which is ideal for the matrix operations used in AI.
- **TPUs** are specialized for machine learning and are highly optimized for tensor operations used in AI models.

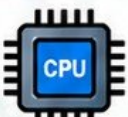
WHY AI NEEDS SPECIALIZED COMPUTE?

- AI models perform millions or billions of matrix calculations (tensor operations).
- These calculations can be parallelized, which makes GPUs and TPUs much faster than CPUs.
- Specialized compute reduces training time, improves performance and handles large models efficiently.

CPU vs GPU vs TPU COMPARISON

FEATURE	CPU	GPU	TPU
Main Purpose	General-purpose computing	Parallel processing (e.g., graphics, AI)	Machine learning (tensor operations)
Architecture	Few powerful cores	Thousands of smaller cores	Specialized tensor cores
Best For	General tasks, system operations, small models	Training and inference of large AI models	Large-scale AI/ML workloads (especially on Google Cloud)
Processing Style	Sequential processing	Massively parallel processing	Highly optimized matrix/tensor processing
Memory	Lower memory bandwidth	High memory bandwidth (HBM/VRAM)	High bandwidth, optimized for AI workloads
Performance in AI	Slower	Fast (widely used)	Very fast (for supported workloads)
Common Use Cases	OS, applications, databases, web servers	Model training, inference, deep learning, data science	Large-scale model training and inference (Google Cloud)

VISUAL COMPARISON



CPU

- Few cores
- General purpose
- Sequential tasks



GPU

- Thousands of cores
- Parallel processing
- Fast for AI workloads



TPU

- Specialized for AI
- Tensor processing
- Very high performance (for supported workloads)

REAL WORLD USE CASES

- **CPU**: Run operating systems, applications, databases.
- **GPU**: Train large language models (LLMs), computer vision models, run inference servers.
- **TPU**: Train and serve large models on Google Cloud (e.g., Gemini, T5, BERT).

KEY TAKEAWAYS

- ✓ CPUs are versatile but slower for AI workloads.
- ✓ GPUs are the most common choice for training and inference.
- ✓ TPUs are highly optimized for machine learning at large scale.
- ✓ AI workloads require specialized compute because they involve massive parallel matrix operations.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

05. GPU FUNDAMENTALS

GPU memory, parallel processing, VRAM, utilization, and basic GPU terminology.

WHAT IS A GPU?

- A **GPU** (Graphics Processing Unit) is a specialized processor designed for **parallel processing**.
- It can execute **thousands** of small tasks at the same time.
- This makes it ideal for **AI and machine learning**, where the same operation is performed on large amounts of data (matrices).

GPU VS CPU (SIMPLIFIED)

CPU



- Few powerful cores
- Good for sequential tasks
- General purpose
- Used in OS, apps, databases, web servers

GPU

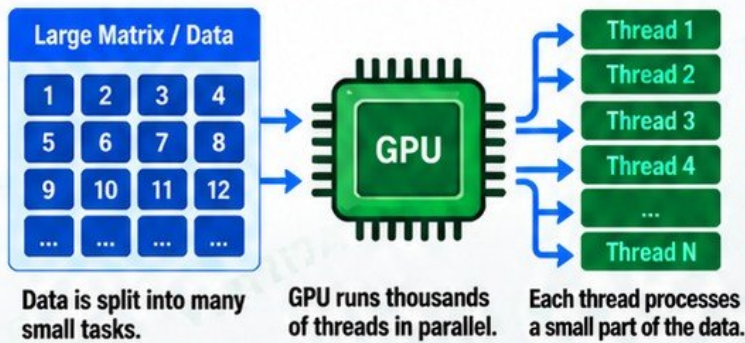


- Thousands of smaller cores
- Good for parallel tasks
- Optimized for AI/ML and graphics
- Handles large matrix operations

WHY GPUS ARE IMPORTANT?

- AI models perform **millions or billions** of calculations.
- GPUs process these calculations **in parallel**, making training and inference much **faster**.
- They reduce training time, improve performance and allow **larger models** to run.

HOW A GPU PROCESSES DATA (PARALLELISM)



COMMON GPU TERMINOLOGY

Term	Meaning
VRAM	Video memory used to store data, model weights and activations.
CUDA	NVIDIA's parallel computing platform.
Tensor Core	Specialized core for matrix operations.
SM	Streaming Multiprocessor, handles many threads.
Warp	Group of threads that execute together.
GPU Utilization	Percentage of time the GPU is busy.
Memory Bandwidth	Speed at which data is read/written to VRAM.
FP32 / FP16 / BF16	Number format used for AI training (affects memory and performance).

GPU MEMORY (VRAM)



- VRAM stores **model weights**, **input data**, **intermediate results** and **activations**.
- Larger models require more VRAM.
- If the model is larger than your VRAM, you may get an **out of memory error (OOM)**.

CHECK GPU INFORMATION

```
$ nvidia-smi
```

- Shows GPU model, VRAM usage, utilization, temperature and more.

```
$ nvidia-smi -l 1
```

- Continuously monitor GPU usage (updates every 1 second).

EXAMPLE OUTPUT (nvidia-smi)

```
+-----+
| GPU   0  NVIDIA A100-SXM4-40GB |
| Utilization   : 45%              |
| Memory Used   : 16384 MiB        |
| Memory Total  : 40960 MiB        |
| Temperature   : 62 C             |
+-----+
```

COMMON GPU ISSUES

- Out of memory (OOM)
- Low GPU utilization
- Incorrect CUDA version
- Driver not installed
- Overheating (high temperature)

TROUBLESHOOTING TIPS

- Check GPU utilization with `nvidia-smi`
- Reduce batch size if OOM occurs
- Use FP16 or BF16 to reduce memory usage
- Ensure correct NVIDIA drivers and CUDA version
- Monitor temperature and system cooling

KEY TAKEAWAYS

- GPUs are built for parallel processing.
- VRAM stores data and model information.
- GPU utilization shows how busy the GPU is.
- Larger models require more VRAM.
- Proper drivers, CUDA and monitoring are important for stable AI workloads.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

06. LINUX FOR AI INFRASTRUCTURE

Understand Linux environments, processes, drivers, permissions, and system resources.

WHY LINUX FOR AI?

- Linux is the **most used** operating system for AI and machine learning workloads.
- It provides **better performance, stability, customization and hardware support** (especially GPUs).
- It is used in servers, cloud platforms, containers and Kubernetes clusters.
- Most AI tools, frameworks and drivers are built and tested on Linux.

LINUX ENVIRONMENTS

-  **Local Machine** (Development)
-  **Remote Server** (Production)
-  **Cloud VM** (AWS, GCP, Azure)
-  **Container** (Docker)
-  **Kubernetes Pod** (Cluster)

KEY COMPONENTS

-  **Processes** (Running programs)
-  **Drivers** (GPU, hardware support)
-  **Permissions** (Users, groups, access control)
-  **System Resources** (CPU, memory, storage, GPU)
-  **File System** (Directories and files)

MANAGE PROCESSES

View, find and manage running processes.

```
# List all processes
ps aux | less

# Find a process by name
ps aux | grep python

# View real-time processes
top

# Kill a process
kill -9 <PID>
```

CHECK SYSTEM RESOURCES

Monitor CPU, memory, storage and GPU usage.

```
# CPU usage
top

# Memory usage
free -h

# Disk usage
df -h

# Detailed system info
htop

# GPU usage (NVIDIA)
nvidia-smi
```

GPU DRIVERS AND TOOLS

Install and manage GPU drivers and tools.

```
# Check NVIDIA GPU
nvidia-smi

# Check CUDA version
nvcc --version

# Check installed driver
cat /proc/driver/nvidia/version

# List GPU devices
lspci | grep -i nvidia

# Check GPU processes
nvidia-smi pmon -c 1
```

USER PERMISSIONS

Manage users, groups and file permissions.

```
# Create a new user
sudo adduser aiuser

# Add user to group (e.g. sudo)
sudo usermod -aG sudo aiuser

# Change file permissions
chmod 755 script.sh

# Change file ownership
chown aiuser:aiuser file.txt

# View current user
whoami
```

COMMON DIRECTORIES

Important Linux directories for AI workloads.

Directory	Purpose
/home	User home directories
/opt	Optional software (e.g. CUDA)
/usr/local	Manually installed software
/var/log	Log files
/etc	Configuration files
/dev	Device files (GPUs, hardware)
/data	Datasets and large files
/mnt	Mounted storage (e.g. cloud)

KEY TAKEAWAYS

- ✓ Linux is the standard environment for AI workloads.
- ✓ Understand and manage processes.
- ✓ Monitor system resources (CPU, memory, storage, GPU).
- ✓ Install and verify GPU drivers.
- ✓ Use proper user permissions and access control.
- ✓ Know key directories and their purpose.
- ✓ These skills are essential for running, troubleshooting and maintaining AI infrastructure.

COMMON ISSUES

- GPU not detected (driver issue)
- Out of memory (OOM)
- Permission denied errors
- Process not found or not running
- High CPU or memory usage
- Disk space full

TROUBLESHOOTING TIPS

- Check nvidia-smi if GPU is not detected.
- Verify driver and CUDA version compatibility.
- Monitor logs in /var/log.
- Ensure correct user permissions.
- Check system resources with top, htop, free -h.
- Restart services or container if needed.

USEFUL TOOLS FOR AI

- nvidia-smi – GPU monitoring
- htop – system monitoring
- nvcc – CUDA compiler
- docker – run AI containers
- kubecti – manage Kubernetes
- git – version control for AI projects

07. AI INFRASTRUCTURE STORAGE

Object storage, block storage, datasets, model artifacts, and checkpoints.

WHAT IS AI STORAGE?

- AI storage holds all the data, models and files used in the AI lifecycle.
- It includes **training datasets**, **model artifacts**, **checkpoints**, logs and application data.
- The storage must be **reliable**, **scalable** and **high performance** to handle large AI workloads.

WHY IT MATTERS?

- AI datasets can be **very large** (TBs or PBs).
- Models and checkpoints need **fast and reliable** storage.
- Poor storage performance **slows down** training and inference.
- The right storage type **reduces costs**, **improves performance** and ensures data durability.

TYPES OF STORAGE



Object Storage

Scalable storage for large unstructured data (e.g. datasets, model artifacts).



Block Storage

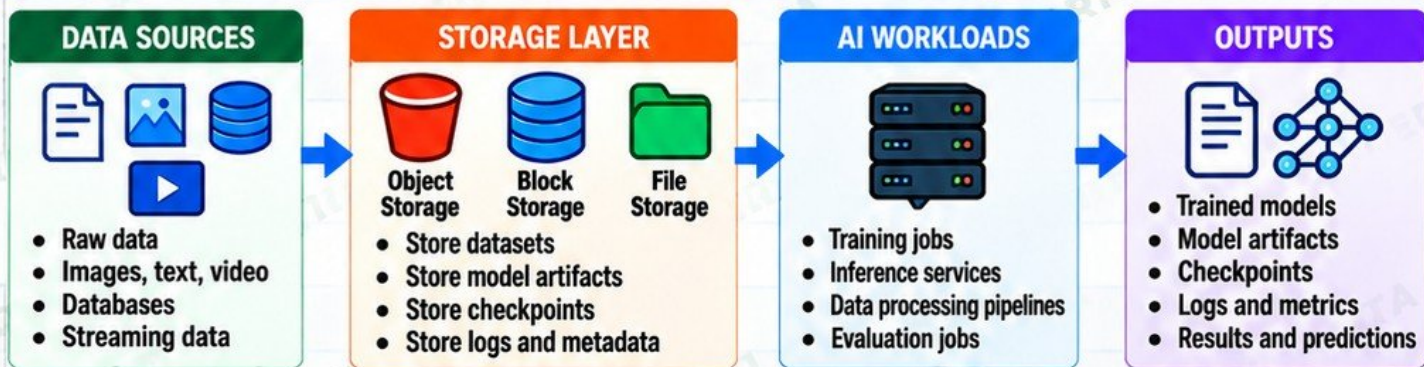
High performance storage for compute (e.g. training jobs, databases).



File Storage

Shared file systems for multiple instances (e.g. distributed training).

AI STORAGE ARCHITECTURE (SIMPLIFIED)



OBJECT STORAGE



- Stores large, unstructured data.
- Highly scalable and durable.
- Used for datasets, model artifacts, backup and logs.
- Examples: AWS S3, Google Cloud Storage, Azure Blob Storage, MinIO, Ceph.

BLOCK STORAGE



- Provides high performance storage for compute instances.
- Used for training jobs, databases and applications.
- Lower latency and higher IOPS than object storage.
- Examples: AWS EBS, Google Persistent Disk, Azure Disk, NVMe SSDs.

FILE STORAGE



- Shared file system accessible by multiple instances.
- Used for distributed training, shared datasets and collaboration.
- Works well with Kubernetes and clusters.
- Examples: NFS, EFS, FSx, Lustre, CephFS, GlusterFS.

DATASETS, MODEL ARTIFACTS AND CHECKPOINTS



DATASETS

- Training data
- Validation data
- Test data
- Stored in object storage (e.g. S3)



MODEL ARTIFACTS

- Trained model files
- Tokenizer files
- Configuration files
- Stored in object storage or model registry



CHECKPOINTS

- Snapshots during training
- Used to resume training
- Stored regularly (e.g. every epoch or time interval)
- Helps prevent data loss



COMMON FILE TYPES

- Datasets: .json, .csv, .parquet
- Models: .pt, .pth, .bin, .safetensors
- Tokenizer: tokenizer.json
- Config: config.json, yaml
- Checkpoints: model.ckpt, step_x
- Logs: .log
- Metrics: .json, .csv

KEY TAKEAWAYS

- Choose the right storage type for each workload.
- Object storage for large datasets and artifacts.
- Block storage for high performance compute.
- File storage for shared access.
- Store checkpoints regularly.
- Use scalable and durable storage.
- Manage cost and performance based on your use case.

08. NETWORKING FOR AI WORKLOADS

Bandwidth, latency, data transfer, and communication between infrastructure components.

WHY NETWORKING MATTERS FOR AI

- AI workloads move large amounts of data between storage, compute, GPUs and services.
- Good networking **improves training speed, inference performance** and overall **system reliability**.
- Poor networking can cause slow training, higher costs and system failures.
- Engineers must understand and design networking for **high bandwidth**, **low latency** and reliable communication.

KEY CONCEPTS



BANDWIDTH

Amount of data that can be transferred per second. (e.g. Gbps)



LATENCY

Time taken for data to travel from source to destination. (e.g. ms)



THROUGHPUT

Actual amount of data successfully transferred over time.



RELIABILITY

Network stability, low packet loss and consistent performance.

COMMON NETWORK COMPONENTS



Switches

Connect multiple devices within a network.



Routers

Route traffic between different networks.



Load Balancers

Distribute traffic across multiple servers.



Firewalls

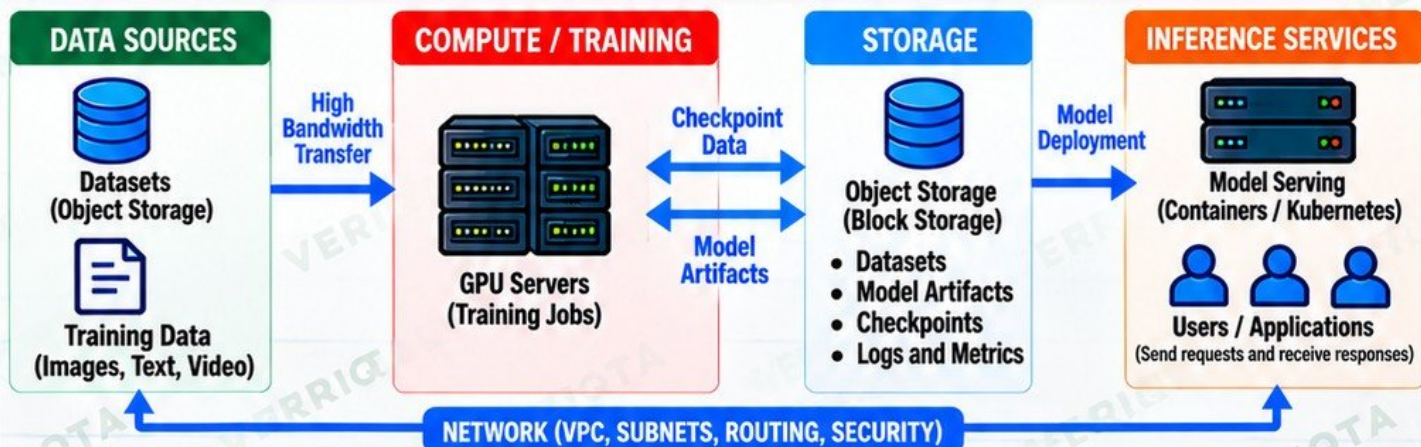
Control and secure network traffic.



VPC / Subnet

Isolated network environment in the cloud.

AI WORKLOAD NETWORK ARCHITECTURE (SIMPLIFIED)



NETWORK REQUIREMENTS FOR AI

- High bandwidth between compute nodes (e.g. 100 Gbps, 200 Gbps, 400 Gbps).
- Low latency for distributed training and inference.
- Reliable and stable connections.
- Support for large data transfers (datasets, model checkpoints).
- Proper network architecture (VPC, subnets, security groups).
- Monitoring and troubleshooting tools.

COMMON ISSUES



- High latency between nodes.
- Low bandwidth causing slow training.
- Packet loss and intermittent disconnections.
- Misconfigured security groups or firewalls.
- Network congestion.
- DNS or routing issues.
- Cross-region data transfer delays.

KEY TAKEAWAYS

- AI workloads require high bandwidth and low latency networking.
- Networking connects all infrastructure components (storage, compute, services).
- Proper network design improves performance, reliability and cost efficiency.
- Monitor network performance and troubleshoot issues.
- Understand cloud networking concepts (VPC, subnets, security groups, load balancers).
- Good networking is essential for scalable and production-ready AI systems.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

09. PYTHON ENVIRONMENTS AND DEPENDENCIES

Virtual environments, package management, reproducibility, and dependency conflicts.

WHY IT MATTERS?

- Keeps project dependencies **isolated**.
- Prevents **version conflicts** between projects.
- Ensures **reproducible** and **consistent** environments.
- Makes **collaboration** and **deployment** easier.
- Essential for **reliable AI** and machine learning workflows.

KEY CONCEPTS



VIRTUAL ENVIRONMENT
Isolated Python environment for a project.



PACKAGE MANAGEMENT
Install, upgrade and manage Python packages.



DEPENDENCIES
Required packages and their versions.



REPRODUCIBILITY
Same environment across different machines.



CONFLICTS
Version mismatches between packages.

COMMON TOOLS



venv Built-in virtual environment tool (from Python).



conda Popular environment and package manager (Anaconda).



pip Install and manage Python packages.



poetry Modern dependency management tool.



pipenv Manages virtual environments and dependencies.



requirements.txt
File to list project dependencies.

CREATE A VIRTUAL ENVIRONMENT

Using venv (Python built-in)

```
# Create a virtual environment
python3 -m venv ai-env

# Activate (Linux/Mac)
source ai-env/bin/activate

# Activate (Windows)
ai-env\Scripts\activate
```



INSTALL AND MANAGE PACKAGES

```
# Install a package
pip install numpy

# Install specific version
pip install torch==2.1.0

# Upgrade a package
pip install --upgrade numpy

# List installed packages
pip list
```

SAVE AND REPRODUCE ENVIRONMENT

```
# Save all dependencies
pip freeze > requirements.txt

# Install from requirements file
pip install -r requirements.txt

# Example requirements.txt
numpy==1.26.4
pandas==2.2.0
torch==2.1.0
scikit-learn==1.4.1
```

CONDA ENVIRONMENT EXAMPLE

```
# Create a conda environment
conda create -n ai-env python=3.11

# Activate environment
conda activate ai-env

# Install packages
conda install pytorch torchvision

# List environments
conda env list
```

COMMON DEPENDENCY CONFLICTS

- ! Different packages require incompatible versions.
- ! Upgrading one package can break another.
- ! Common in AI/ML due to complex dependencies (e.g. torch, cuda, transformers).
- ! Use virtual environments and version pinning to avoid conflicts.

BEST PRACTICES

- ✓ Always use a virtual environment.
- ✓ Pin package versions.
- ✓ Keep a requirements.txt file.
- ✓ Use conda for complex AI/ML projects (with GPU libraries).
- ✓ Regularly update and test dependencies.
- ✓ Avoid installing packages globally.
- ✓ Document your environment for reproducibility.

USEFUL COMMANDS



- python --version (check Python version)
- pip --version (check pip version)
- pip list (list installed packages)
- pip show <package> (package details)
- pip check (check for broken dependencies)

REAL WORLD AI/ML PACKAGES



- numpy (numerical computing)
- pandas (data processing)
- scikit-learn (machine learning)
- torch (deep learning)
- tensorflow (deep learning)
- transformers (LLMs and NLP)

KEY TAKEAWAYS



- Virtual environments keep projects isolated and clean.
- Package management ensures the right versions.
- Reproducibility is critical for AI/ML.
- Dependency conflicts are common but can be managed with best practices.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

10. CONTAINERS FOR AI APPLICATIONS

Package AI applications using Docker, images, containers, and persistent storage.

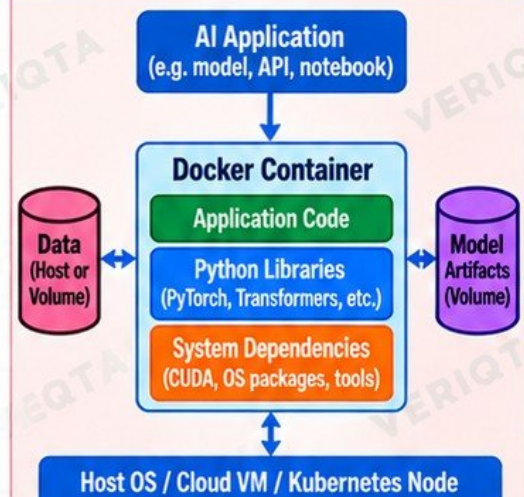
WHAT ARE CONTAINERS?

- Containers package an application and all its dependencies into a single, **portable unit**.
- They ensure **consistency** across different environments.
- Ideal for **AI applications** because they include **exact Python libraries**, **CUDA drivers**, and system tools.
- Run the same container on your laptop, a server or in the cloud with the same behaviour.

KEY CONCEPTS

-  **Dockerfile**
Instructions to build an image.
-  **Image**
Read-only template with the application and dependencies.
-  **Container**
Running instance of an image.
-  **Volume**
Persistent storage for data and model files.
-  **Network**
Allows containers to communicate.
-  **Registry**
Store and share images (e.g. Docker Hub, ECR, GCR).

CONTAINER ARCHITECTURE (SIMPLIFIED)



SIMPLE DOCKERFILE FOR AN AI APP

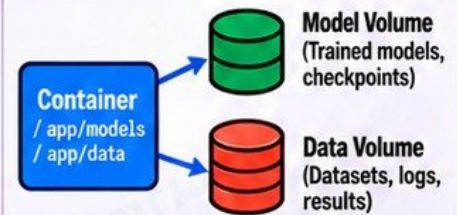
```
# Use official PyTorch image
FROM pytorch/pytorch:2.1.0-cuda12.1-cudnn8-runtime
# Set working directory
WORKDIR /app
# Copy requirements and install
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
# Copy application code
COPY . .
# Expose port (e.g. for API)
EXPOSE 8000
# Run the application
CMD ["python", "app.py"]
```



BUILD AND RUN CONTAINER

```
# Build the image
docker build -t ai-app:1.0 .
# Run the container
docker run -d \
  --name ai-app \
  --gpus all \
  -p 8000:8000 \
  -v /data/models:/app/models \
  -v /data/datasets:/app/data \
  ai-app:1.0
# Check running containers
docker ps
# View logs
docker logs -f ai-app
```

PERSISTENT STORAGE (VOLUMES)



- Use Docker volumes to keep data and models even if the container is deleted.
- You can also mount host directories.
- Essential for large datasets and trained model artifacts.

USEFUL DOCKER COMMANDS



- docker build -t <name>**
Build an image
- docker run <image>**
Run a container
- docker ps**
List running containers
- docker images**
List images
- docker stop <id>**
Stop a container
- docker rm <id>**
Remove a container

AI CONTAINER EXAMPLE

Run a Jupyter Notebook with GPU support.

```
docker run -d \
  --name ai-notebook \
  --gpus all \
  -p 8888:8888 \
  -v /data/notebooks:/workspace \
  pytorch/pytorch:2.1.0-cuda12.1-cudnn8
jupyter lab --ip=0.0.0.0 \
  --allow-root --no-browser \
  --NotebookApp.token=''
```

COMMON USE CASES

-  **Model Training**
Containers with GPU and all dependencies.
-  **Model Inference**
Lightweight containers for serving models.
-  **Data Processing**
Containers for data preparation and feature engineering.
-  **AI Services**
Run APIs, notebooks and pipelines in containers.

KEY TAKEAWAYS

- ✓ Containers package your AI application and dependencies.
- ✓ Use Dockerfiles to create reproducible images.
- ✓ Use volumes for persistent storage (datasets, models).
- ✓ Run containers with GPU support for AI workloads.
- ✓ Containers improve portability, consistency and collaboration.
- ✓ Essential for training, inference and production AI systems.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

11. KUBERNETES FOR AI WORKLOADS

Pods, Deployments, Jobs, resource requests, limits, and basic GPU scheduling.

WHAT IS KUBERNETES?

- Kubernetes is a **container orchestration platform** for running, scaling and managing containerized applications.
- It **automates** deployment, scaling, networking and self-healing.
- Ideal for AI workloads because it can run **training jobs**, **inference services** and **data pipelines** at scale.
- Supports **GPU scheduling**, resource management and high availability.

KEY KUBERNETES COMPONENTS



Pod

Smallest unit. Runs one or more containers.



Deployment

Manages replicas of your application (e.g. inference API).



Job

Runs a task to completion (e.g. model training).



Service

Exposes your application inside or outside the cluster.



Ingress

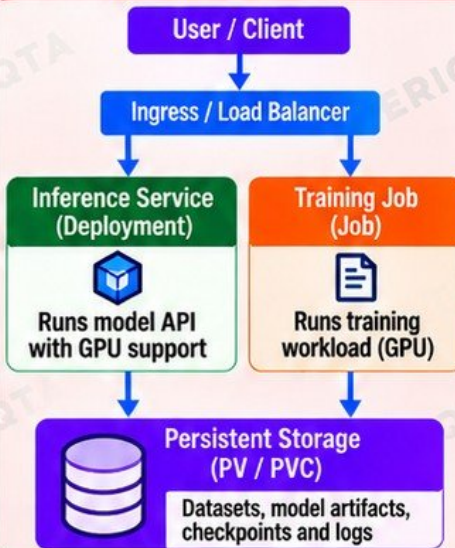
Manages external access (HTTP/HTTPS).



Persistent Volume (PV)

Provides persistent storage for datasets and model files.

AI WORKLOADS ON KUBERNETES



EXAMPLE: AI INFERENCE DEPLOYMENT (WITH GPU)

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: ai-inference
spec:
  replicas: 1
  selector:
    matchLabels:
      app: ai-inference
  template:
    metadata:
      labels:
        app: ai-inference
    spec:
      containers:
        - name: app
          image: my-ai-image:latest
          resources:
            requests:
              nvidia.com/gpu: 1
              cpu: "4"
              memory: "16Gi"
            limits:
              nvidia.com/gpu: 1
              cpu: "8"
              memory: "32Gi"
          ports:
            - containerPort: 8000
  
```



EXAMPLE: AI TRAINING JOB (WITH GPU)

```

apiVersion: batch/v1
kind: Job
metadata:
  name: model-training
spec:
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: trainer
          image: my-trainer:latest
          command: ["python", "train.py"]
          resources:
            requests:
              nvidia.com/gpu: 1
              cpu: "8"
              memory: "32Gi"
            limits:
              nvidia.com/gpu: 1
              cpu: "8"
              memory: "32Gi"
          volumeMounts:
            - name: data
              mountPath: /data
      volumes:
        - name: data
          persistentVolumeClaim:
            claimName: ai-data-pvc
  
```



RESOURCE REQUESTS AND LIMITS

Requests



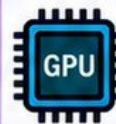
- Guarantee minimum resources for your container.
- Helps Kubernetes schedule pods on suitable nodes.
- Example: cpu, memory, nvidia.com/gpu.

Limits



- Set the maximum resources your container can use.
- Prevents a container from using all node resources.
- Important for stable cluster performance.

GPU SCHEDULING



- Use NVIDIA GPU Operator or device plugins.
- Request GPUs using nvidia.com/gpu.
- Kubernetes schedules the pod to nodes with available GPUs.
- Supports multiple GPU types (e.g. A10, A100, H100).

USEFUL KUBECTL COMMANDS



- kubectl get nodes
- kubectl get pods
- kubectl get deployments
- kubectl get jobs
- kubectl describe pod <name>
- kubectl logs <pod>
- kubectl get pvc
- kubectl apply -f <file.yaml>
- kubectl delete <resource> <name>

PERSISTENT STORAGE FOR AI



- Use Persistent Volumes (PV) and Persistent Volume Claims (PVC).
- Store datasets, model artifacts, checkpoints and logs.
- Supports cloud storage (e.g. EBS, GCP Persistent Disk, NFS, S3 via CSI).
- Keeps data even if the pod is deleted.
- Essential for training and large datasets.

KEY TAKEAWAYS



- Kubernetes helps run AI workloads efficiently at scale.
- Use Deployments for inference and Jobs for training.
- Set resource requests and limits.
- Use GPU scheduling with nvidia.com/gpu.
- Use persistent storage for datasets and model artifacts.
- Monitor and optimize cluster resources.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

13. INTRODUCTION TO MLOps

Model versioning, experiment tracking, reproducibility, and operational workflows.

WHAT IS MLOps?

- MLOps (Machine Learning Operations) is a set of practices to **build, deploy, monitor and maintain** machine learning models in production.
- Combines **machine learning, DevOps and data engineering**.
- Ensures models are **reproducible, scalable, reliable** and easy to manage.
- Helps teams move from experiments to **real-world production systems**.

KEY CONCEPTS



MODEL VERSIONING

Track and manage different versions of models.



EXPERIMENT TRACKING

Log parameters, metrics and artifacts from training experiments.



REPRODUCIBILITY

Ensure the same results can be reproduced across environments.



OPERATIONAL WORKFLOWS

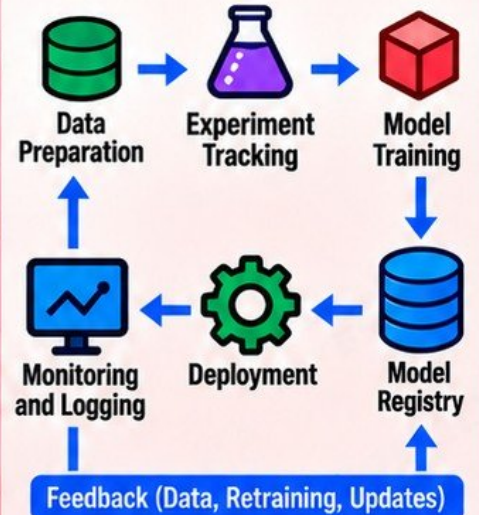
Automate training, testing, deployment and monitoring.



COLLABORATION

Enable teams to work together efficiently.

MLOps LIFECYCLE (SIMPLIFIED)



MODEL VERSIONING



v1.0 → Initial model (live)



v1.1 → Improved model (better accuracy)



v1.2 → Experiment model (under testing)



v2.0 → Major new version (production)

EXPERIMENT TRACKING



- Track hyperparameters, metrics and model artifacts.
- Compare different experiments.
- Understand what worked and why.
- Reproduce results easily.

```
# Log parameters and metrics (MLflow)
import mlflow
mlflow.log_param("learning_rate", 0.001)
mlflow.log_param("batch_size", 32)
mlflow.log_metric("accuracy", 0.92)
mlflow.log_artifact("model.pkl")
```

MLOps TOOLCHAIN (EXAMPLES)



MLflow

Experiment tracking, model registry and lifecycle management.



DVC

Data versioning for machine learning projects.



Kubeflow

End-to-end ML workflows on Kubernetes.



TensorBoard

Visualize training metrics and model performance.



Airflow

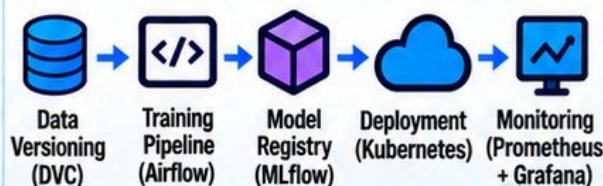
Schedule and automate ML pipelines.

REPRODUCIBILITY



- Use version control (Git) for code.
- Track data versions (DVC).
- Record dependencies (requirements.txt or Conda).
- Use containers (Docker).
- Log experiments (MLflow).
- Set random seeds for consistent results.

MLOps WORKFLOW EXAMPLE



KEY TAKEAWAYS



- MLOps helps you build and maintain reliable ML systems.
- Track model versions and experiments.
- Ensure reproducibility across environments.
- Automate training, deployment and monitoring.
- Use the right tools for your workflow.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

14. AI TRAINING INFRASTRUCTURE

Understand training jobs, datasets, accelerators, checkpoints, and resource allocation.

WHAT IS AI TRAINING INFRASTRUCTURE?

- The **compute, storage, network** and software systems used to train machine learning and deep learning models.
- Involves **datasets, accelerators** (GPUs/TPUs), distributed training, **checkpoints** and efficient **resource allocation**.
- Designed to handle large datasets, long-running training jobs and high computational demands.
- Must be **reliable, scalable** and **cost-efficient**.

KEY COMPONENTS



DATASETS

Training, validation and test data. Stored on local storage or object storage (e.g. S3).



ACCELERATORS

GPUs, TPUs or specialized hardware for faster training.



TRAINING JOBS

Processes that run model training (single or distributed).



CHECKPOINTS

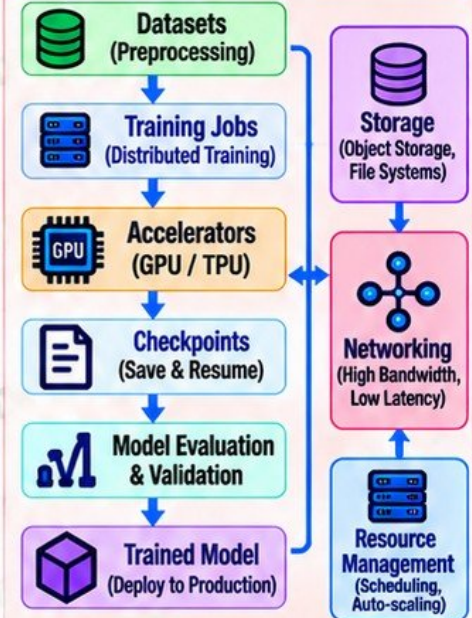
Saved model state during training for recovery and resuming.



RESOURCE ALLOCATION

Allocate and manage compute, memory, storage and network resources efficiently.

AI TRAINING INFRASTRUCTURE FLOW



EXAMPLE: TRAINING JOB (PyTorch)

```

import torch
from torch.utils.data import DataLoader

# Initialize model and move to GPU
model = MyModel().to("cuda")

# Load dataset
train_loader = DataLoader(dataset,
                          batch_size=32,
                          shuffle=True)

# Training loop
for epoch in range(epochs):
    for data, labels in train_loader:
        data, labels = data.to("cuda"), labels.to("cuda")
        outputs = model(data)
        loss = loss_fn(outputs, labels)
        loss.backward()
        optimizer.step()
        optimizer.zero_grad()
  
```



DISTRIBUTED TRAINING (MULTI-GPU)

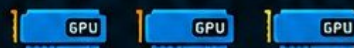
```

# Using PyTorch Distributed Data Parallel
import torch
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as DDP

# Initialize process group
dist.init_process_group(backend="nccl")

# Set device
local_rank = int(os.environ["LOCAL_RANK"])
torch.cuda.set_device(local_rank)

# Wrap model
model = MyModel().to(local_rank)
model = DDP(model, device_ids=[local_rank])
  
```



CHECKPOINTS (SAVE & RESUME)

```

# Save checkpoint
checkpoint = {
    "epoch": epoch,
    "model_state": model.state_dict(),
    "optimizer_state": optimizer.state_dict(),
}
torch.save(checkpoint, "checkpoint.pth")

# Load checkpoint
checkpoint = torch.load("checkpoint.pth")
model.load_state_dict(checkpoint["model_state"])
optimizer.load_state_dict(checkpoint["optimizer_state"])
start_epoch = checkpoint["epoch"] + 1
  
```



RESOURCE ALLOCATION



- Allocate resources based on model size, dataset size and training time.
- Use resource managers (Kubernetes, Slurm) for scheduling.
- Monitor resource usage (GPU, CPU, memory, network, storage).
- Scale resources based on workload (auto-scaling).

COMMON CHALLENGES

- Large dataset storage and access.
- Long training times.
- GPU/TPU availability and high costs.
- Handling distributed training failures.
- Checkpointing and recovery.
- Efficient resource allocation.
- Network bandwidth and latency.
- Monitoring and troubleshooting training jobs.

KEY TAKEAWAYS

- AI training infrastructure requires compute, storage, network and software.
- Use GPUs/TPUs for faster training.
- Store and manage datasets efficiently.
- Use checkpoints to save and resume training.
- Allocate resources based on workload demands.
- Distributed training enables larger and faster model training.
- Monitor resource usage and optimize for cost and performance.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

15. MODEL SERVING AND INFERENCE

How deployed models receive requests and generate predictions or responses.

WHAT IS MODEL SERVING?

- Model serving makes a trained model available to receive **real-time** or **batch requests** and generate predictions or responses.
- It exposes the model through an **API** (e.g. REST or gRPC).
- Used for **AI applications** such as chatbots, recommendations, image analysis, and more.
- Must be **scalable, reliable, secure** and **low latency**.

KEY COMPONENTS



TRAINED MODEL

Model file and dependencies (e.g. .pt, .onnx, .bin).



SERVING FRAMEWORK

Tools to serve the model (e.g. FastAPI, TorchServe, Triton Inference Server).



INFERENCE API

Endpoint that receives requests and returns predictions.



INFRASTRUCTURE

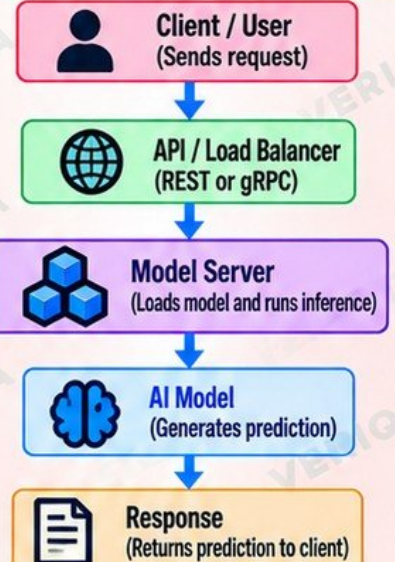
Runs the model (CPU, GPU, container, Kubernetes, etc).



MONITORING

Track performance, latency, errors and usage.

MODEL SERVING FLOW



SIMPLE INFERENCE API (FASTAPI)

```

from fastapi import FastAPI
from pydantic import BaseModel
import torch

app = FastAPI()
model = torch.load("model.pt")
model.eval()

class InputData(BaseModel):
    text: str

@app.post("/predict")
def predict(data: InputData):
    with torch.no_grad():
        result = model(data.text)
    return {"prediction": str(result)}
  
```



CONTAINERIZED MODEL SERVER

```

# Use official Python image
FROM python:3.11-slim
# Set working directory
WORKDIR /app
# Copy requirements and install
COPY requirements.txt .
RUN pip install --no-cache-dir \
    -r requirements.txt
# Copy model and app code
COPY model.pt .
COPY app.py .
# Expose port
EXPOSE 8000
# Run the API
CMD ["uvicorn", "app:app", \
    "--host", "0.0.0.0", \
    "--port", "8000"]
  
```



DEPLOYMENT OPTIONS



Single Server

Run on a VM or physical server.



Docker Container

Package the model and dependencies.



Kubernetes

Deploy and scale using Kubernetes.



GPU Inference

Use GPUs for faster inference (e.g. NVIDIA, CUDA, TensorRT).



Cloud Services

Use managed services (e.g. AWS SageMaker, GCP Vertex AI, Azure ML).

REAL WORLD EXAMPLE



Example Request (cURL)

```

curl -X POST http://localhost:8000/predict \
-H "Content-Type: application/json" \
-d '{"text": "Explain Kubernetes"}'
  
```

Example Response

```

{
  "prediction": "Kubernetes is a container  
orchestration platform that ..."
}
  
```

COMMON CHALLENGES

- High latency under load.
- Scaling to handle many requests.
- Managing large models (e.g. LLMs).
- GPU resource allocation.
- Keeping models updated.
- Monitoring errors and drift.
- Cost and infrastructure management.

KEY TAKEAWAYS

- Model serving exposes trained models as APIs.
- Use containers and Kubernetes for scalability.
- Choose the right hardware (CPU/GPU) for your model.
- Monitor performance, latency and errors.
- Keep models secure and updated.
- Use the right serving framework for your use case.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

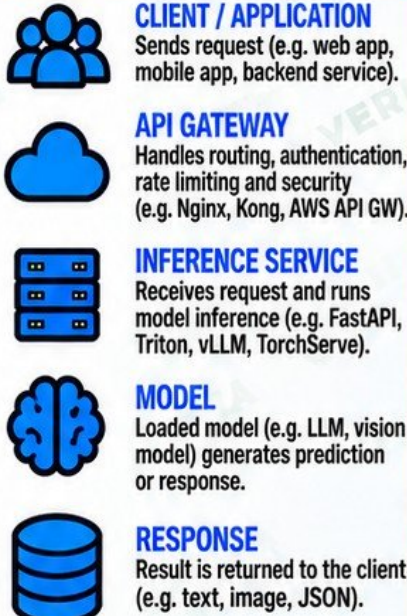
16. AI APIs AND REQUEST FLOW

Trace an application request through an API, inference service, model, and response.

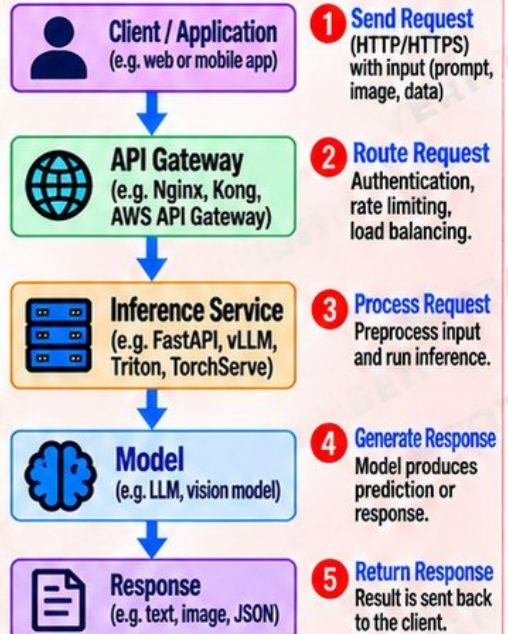
WHAT ARE AI APIS?

- AI APIs allow applications to **send a request** (input) to a model and **receive a response** (output) over HTTP.
- They **hide the complexity** of model serving and infrastructure.
- Used for **chat**, **text generation**, **image generation**, **embeddings**, **classification** and more.
- Can be public (e.g. OpenAI API) or private (your own model serving infrastructure).
- Must be **secure**, **scalable** and have **low latency**.

KEY COMPONENTS



AI API REQUEST FLOW (SIMPLIFIED)



EXAMPLE: API REQUEST (PYTHON)

```
import requests
url = "http://localhost:8000/v1/predict"
headers = {"Content-Type": "application/json"}

payload = {
    "prompt": "Explain Kubernetes in simple terms"
}

response = requests.post(url,
                        json=payload,
                        headers=headers)

print(response.status_code)
print(response.json())
```

SIMPLE FASTAPI INFERENCE API

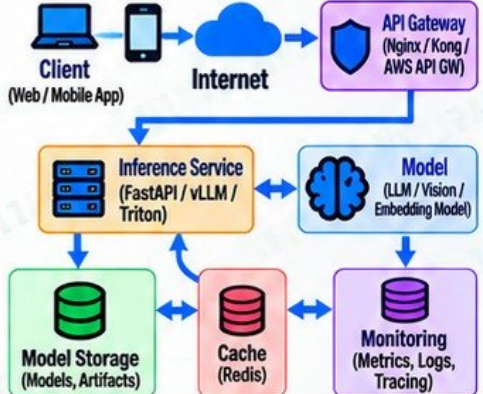
```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class RequestData(BaseModel):
    prompt: str

@app.post("/v1/predict")
def predict(data: RequestData):
    # Load model and generate response
    result = model.generate(data.prompt)
    return {"response": result}
```

REAL WORLD ARCHITECTURE



EXAMPLE: CURL REQUEST

```
curl -X POST http://localhost:8000/v1/predict \
-H "Content-Type: application/json" \
-d '{
    "prompt": "Summarize machine learning \
in one paragraph"
}'
```

COMMON RESPONSE (JSON)

```
{
  "response": "Machine learning is a subset of AI that allows computers to learn patterns from data and make predictions without being explicitly programmed."
}
```

COMMON USE CASES

- Chat and Q&A
- Text generation
- Image generation
- Embeddings and search
- Classification and analysis

KEY TAKEAWAYS

- Trace the full request flow from client to model and back.
- APIs make it easy to integrate AI into applications.
- Use authentication, rate limits and monitoring.
- Choose the right inference framework for your use case.
- Ensure low latency, scalability and reliability.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

17. SCALING AI INFRASTRUCTURE

Understand replicas, batching, autoscaling, GPU capacity, and basic scaling trade-offs.

WHY SCALING AI INFRASTRUCTURE?

- Handle **more users** and **higher request traffic**.
- Reduce **latency** and improve **response time**.
- Use **replicas**, **batching** and **autoscaling** to efficiently serve models.
- Manage **GPU capacity** and control **costs**.
- Understand the trade-offs between **performance**, **cost** and **reliability**.

KEY SCALING CONCEPTS



REPLICAS

Run multiple instances of your model to handle more traffic.



BATCHING

Process multiple requests together to improve GPU utilization and throughput.



AUTOSCALING

Automatically increase or decrease replicas based on demand (CPU, GPU or custom metrics).



GPU CAPACITY

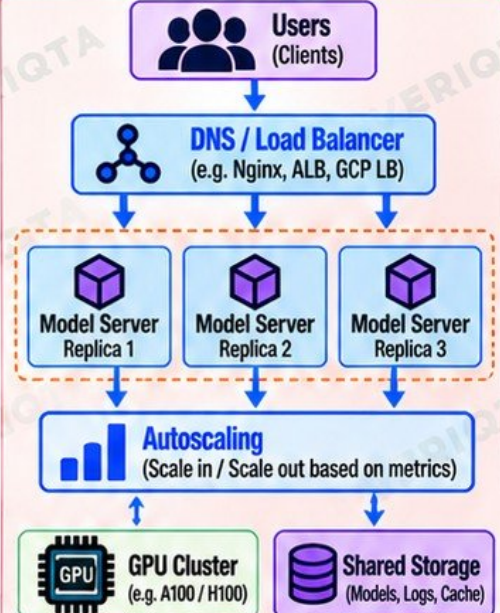
Choose the right GPU type and size for your workload (e.g. A10, A100, H100).



SCALING TRADE-OFFS

Balance cost, performance, latency and reliability.

SCALING ARCHITECTURE (EXAMPLE)



EXAMPLE: REPLICAS (KUBERNETES)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: model-server
spec:
  replicas: 3
  selector:
    matchLabels:
      app: model-server
  template:
    metadata:
      labels:
        app: model-server
    spec:
      containers:
        - name: server
          image: my-ai-model:latest
          resources:
            limits:
              nvidia.com/gpu: 1
          ports:
            - containerPort: 8000
```



EXAMPLE: BATCHING (PYTHON)

```
from transformers import pipeline
import torch

pipe = pipeline(
    "text-generation",
    model="meta-llama/Llama-2-7b-hf",
    device="cuda",
    batch_size=8
)

prompts = [
    "Explain AI in simple terms.",
    "What is Kubernetes?",
    "Write a short poem about clouds."
]

results = pipe(prompts,
               max_new_tokens=50,
               batch_size=8)

for i, r in enumerate(results):
    print(f"[{i}] {r['generated_text']}")
```



AUTOSCALING (KUBERNETES HPA)

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: model-server-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: model-server
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: nvidia.com/gpu
        target:
          type: Utilization
          averageUtilization: 70
```



GPU CAPACITY EXAMPLE

GPU	VRAM	Best For	Example Use Case
A10	24 GB	Inference	Chatbots, small LLMs
A100	40/80 GB	Training & Inference	LLMs, vision models
H100	80 GB	Large Training	Large LLMs, multi-node

SCALING TRADE-OFFS

- ↑ More replicas → Higher cost
- ↑ Larger batch size → Higher throughput but slightly higher latency
- ↑ Autoscaling → Better resource usage but requires monitoring
- ↓ Higher GPU capacity → Higher cost but faster training/inference
- ⚖️ Need to balance cost, latency, performance and reliability

KEY TAKEAWAYS

- ✓ Use replicas to handle more traffic.
- ✓ Use batching to improve GPU utilization.
- ✓ Implement autoscaling based on real metrics.
- ✓ Choose the right GPU capacity for your workload.
- ✓ Understand and manage scaling trade-offs.
- ✓ Monitor performance, cost and reliability.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

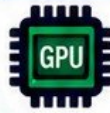
18. AI INFRASTRUCTURE OBSERVABILITY

Monitor GPU utilization, memory, latency, throughput, logs, errors, and service health.

WHY OBSERVABILITY FOR AI?

- Ensures AI services are **reliable, performant** and **available**.
- Helps you **detect** and **troubleshoot issues** quickly.
- Monitor **GPU utilization, memory, latency, throughput, logs** and **errors**.
- Understand **system health** across the full stack (infrastructure, models and services).
- Essential for maintaining **high-quality** and **cost-efficient AI systems** in production.

KEY OBSERVABILITY COMPONENTS



GPU MONITORING

Track GPU utilization, memory, temperature and health.



SYSTEM METRICS

Monitor CPU, memory, disk, network and storage.



APPLICATION METRICS

Track latency, throughput, request rate and error rate.



LOGS

Collect and analyze logs from applications, containers and infrastructure.



ERROR TRACKING

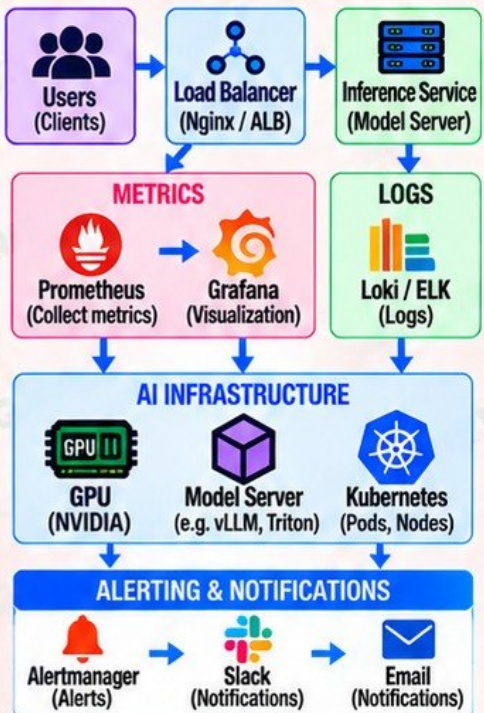
Identify and alert on failures, exceptions and anomalies.



SERVICE HEALTH

Monitor the health and availability of all components (models, APIs, databases, etc).

AI OBSERVABILITY ARCHITECTURE



GPU MONITORING (NVIDIA)

```

# Check GPU status
nvidia-smi

# Continuous monitoring
watch -n 2 nvidia-smi

# Show detailed GPU info
nvidia-smi --query-gpu=
name,utilization.gpu,memory.used,
memory.total,temperature.gpu \
--format=csv

# Monitor running processes
nvidia-smi pmon -c 1
    
```



APPLICATION METRICS (PROMETHEUS)

```

from prometheus_client import
Counter, Histogram, start_http_server

# Metrics
REQUEST_COUNT = Counter(
    "ai_requests_total",
    "Total number of inference
requests")
REQUEST_LATENCY = Histogram(
    "ai_request_latency_seconds",
    "Inference request latency"
)

# Start metrics server
start_http_server(8000)
    
```



LOGGING (PYTHON)

```

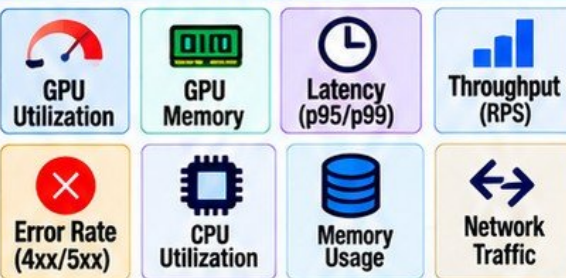
import logging
from datetime import datetime

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s [%(levelname)s]
%(name)s: %(message)s",
    handlers=[
        logging.StreamHandler(),
        logging.FileHandler("app.log")
    ]
)

logger = logging.getLogger("ai_service")
logger.info("Model loaded successfully")
logger.error("Inference failed: timeout")
    
```



KEY METRICS TO MONITOR



COMMON ALERTS

- High GPU utilization (> 90%).
- High GPU memory usage.
- Increased latency (p95/p99).
- High error rate (4xx/5xx).
- Model server down or unhealthy.
- Container/pod crashes.
- Disk space running low.
- Unusual drop in throughput.

KEY TAKEAWAYS

- Monitor GPU, memory, latency, throughput, logs and errors.
- Use Prometheus + Grafana for metrics.
- Collect and centralize logs (Loki/ELK).
- Set up alerts for key failures and performance issues.
- Monitor the health of all components (models, APIs, infrastructure).
- Observability helps you ensure reliable, performant and cost-efficient AI systems.

19. AI INFRASTRUCTURE SECURITY

Protect model artifacts, datasets, credentials, containers, APIs, and infrastructure access.

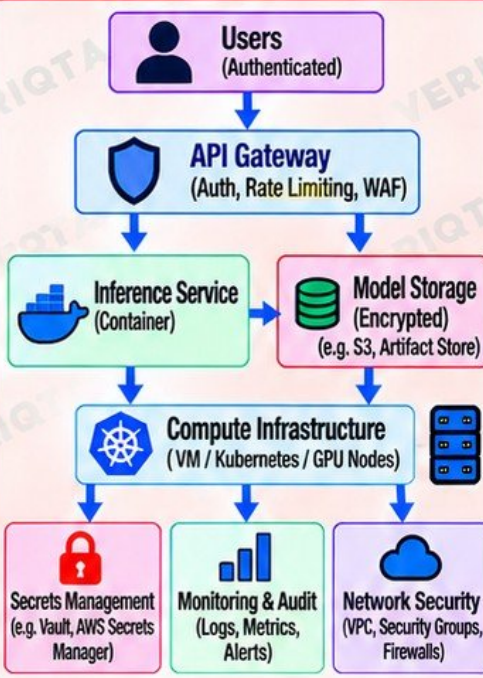
WHY SECURITY FOR AI INFRASTRUCTURE?

- Protect **sensitive data** such as datasets, model artifacts and user information.
- Prevent **unauthorized access** to infrastructure, models and APIs.
- Reduce the risk of **data leaks, model theft** and misuse.
- Ensure **integrity, availability and reliability** of AI services.
- Meet **compliance and regulatory requirements** (e.g. GDPR, HIPAA).

KEY SECURITY COMPONENTS

- IDENTITY & ACCESS MANAGEMENT**
Control who can access systems, data and models (e.g. IAM, RBAC, SSO).
- DATA SECURITY**
Protect datasets with encryption, access controls and data masking.
- MODEL SECURITY**
Secure model artifacts, prevent tampering and unauthorized access (e.g. signing, checksums).
- CONTAINER SECURITY**
Use minimal images, scan for vulnerabilities and follow best practices.
- API SECURITY**
Use authentication, rate limiting and input validation.
- INFRASTRUCTURE SECURITY**
Secure cloud resources, network access, secrets and monitoring.

AI INFRASTRUCTURE ARCHITECTURE



EXAMPLE: IAM POLICY (AWS)

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3::ai-models/*"
    }
  ]
}
  
```



EXAMPLE: SECURE CONTAINER (DOCKER)

```

# Use minimal base image
FROM python:3.11-slim

# Create non-root user
RUN useradd -m appuser
USER appuser

# Copy application files
COPY --chown=appuser:appuser . /app
WORKDIR /app

# Install dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Run the application
CMD ["python", "app.py"]
  
```



EXAMPLE: API SECURITY (FASTAPI)

```

from fastapi import FastAPI, Depends, HTTPException
from fastapi.security import APIKeyHeader

app = FastAPI()

api_key_header = APIKeyHeader(name="X-API-Key")

async def verify_api_key(key: str = Depends(api_key_header)):
    if key != "your-secret-key":
        raise HTTPException(status_code=403, detail="Invalid API Key")
    return key

@app.post("/predict")
async def predict(data: dict, key: str = Depends(verify_api_key)):
    # Add input validation and rate limiting
    return {"prediction": "result"}
  
```

KEY SECURITY CONTROLS

- Access Control (IAM / RBAC)
- Encryption (At Rest & In Transit)
- Secrets Management (e.g. Vault)
- Image Scanning (Trivy, ECR)
- Network Security (VPC, Firewalls)
- Input Validation (API Security)
- Audit & Logging (CloudTrail, ELK)
- Compliance (GDPR, HIPAA)

COMMON THREATS

- Unauthorized access to models and datasets.
- Data leakage (sensitive or proprietary data).
- Exposed credentials and secrets.
- Vulnerable containers and images.
- Unsecured APIs (no auth or rate limiting).
- Model tampering or poisoning.
- Insecure infrastructure configuration.
- Lack of monitoring and audit logs.

KEY TAKEAWAYS

- Use strong identity and access management.
- Encrypt datasets and model artifacts.
- Keep credentials in a secrets manager.
- Use minimal and secure container images.
- Implement API authentication and rate limiting.
- Secure your cloud and network infrastructure.
- Monitor logs, metrics and audit trails.
- Follow compliance and security best practices.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

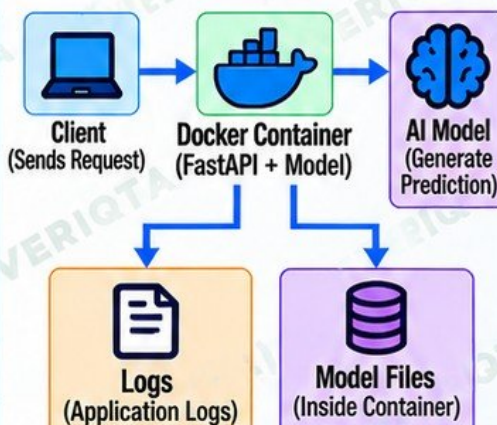
20. PRACTICAL LAB: DEPLOY AN AI SERVICE

Build a small containerized inference API, run it locally, inspect logs, test requests, and troubleshoot failures.

WHAT YOU WILL DO

- ✓ Build a simple AI inference API using Python (FastAPI).
- ✓ Containerize the application with Docker.
- ✓ Run the service locally.
- ✓ Test requests and view responses.
- ✓ Inspect logs and monitor the service.
- ✓ Simulate and troubleshoot common failures.

LAB ARCHITECTURE



PROJECT STRUCTURE

```

ai-service-lab/
├── app/
│   ├── app.py
│   ├── model.py
│   └── requirements.txt
├── Dockerfile
├── .dockerignore
├── test/
│   └── test_api.py
└── README.md
  
```

1. SIMPLE FASTAPI APP (app.py)

```

from fastapi import FastAPI
from pydantic import BaseModel
import uvicorn

app = FastAPI()

class RequestData(BaseModel):
    text: str

@app.post("/predict")
def predict(data: RequestData):
    # Simple mock AI response
    result = f"AI Response: {data.text}"
    return {"prediction": result}

if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=8000)
  
```



2. DOCKERFILE

```

FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir \
    -r requirements.txt

COPY app/ ./app/

EXPOSE 8000

CMD ["uvicorn", "app.app:app", \
    "--host", "0.0.0.0", \
    "--port", "8000"]
  
```



3. BUILD AND RUN CONTAINER

```

# Build the Docker image
docker build -t ai-service:latest .

# Run the container
docker run -d --name ai-service \
    -p 8000:8000 ai-service:latest

# Check if running
docker ps

# View logs
docker logs -f ai-service
  
```



4. TEST THE API

```

# Test with curl
curl -X POST http://localhost:8000/predict \
    -H "Content-Type: application/json" \
    -d '{"text": "Explain Kubernetes in simple terms"}'

# Expected response
{
  "prediction": "AI Response: Explain
    Kubernetes in simple terms"
}
  
```



5. VIEW LOGS

```

# Follow application logs
docker logs -f ai-service

# Example log output
INFO: Uvicorn running on
INFO: Application startup complete
INFO: POST /predict 200 OK
  
```



6. SIMULATE A FAILURE

```

# Stop the container
docker stop ai-service

# Run with wrong port (simulate issue)
docker run -d --name ai-service \
    -p 9999:8000 ai-service:latest

# Check logs for errors
docker logs -f ai-service
  
```



7. TROUBLESHOOTING TIPS

- ✓ Check container logs for errors.
- ✓ Verify the container is running (docker ps).
- ✓ Ensure the correct port is exposed.
- ✓ Check API request format (JSON).
- ✓ Validate model files and dependencies.
- ✓ Monitor resource usage (CPU, memory).

COMMON ISSUES AND FIXES

Issue	Reason	Fix
Container not running	Build or runtime error	Check logs and rebuild
Port already in use	Another service using the port	Use different port (e.g. 8001)
API returns error	Invalid request format	Check JSON structure
Model not loading	Missing files or dependencies	Verify files and requirements

KEY TAKEAWAYS

- ✓ You can containerize and run an AI inference API locally.
- ✓ Docker makes it easy to package models and dependencies.
- ✓ You can test, inspect logs and troubleshoot failures.
- ✓ The same approach can be used in production with Kubernetes, autoscaling and monitoring.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta